



# Architecture Système V2.0

Plateforme SaaS multi-tenant d'agriculture de précision. Conception technique complète, stratégie de gating Standard / Premium, et décisions architecturales.

PRODUIT

LeadFarm Precision Ag

PILOTE

Domaine Khelifa · SBA

PATTERN

Modular Monolith + Edge

PLANS

Standard / Premium

# Sommaire

---

|    |                                                  |    |
|----|--------------------------------------------------|----|
| 01 | Contexte & objectifs architecturaux .....        | 3  |
| 02 | Vue d'ensemble — les 6 couches .....             | 4  |
| 03 | Stratégie multi-tenant & isolation .....         | 5  |
| 04 | Plan Gate — le keystone Standard / Premium ..... | 6  |
| 05 | Couche données & modèle pivot .....              | 8  |
| 06 | Services métier (Edge Functions) .....           | 10 |
| 07 | Flux de données critiques .....                  | 11 |
| 08 | Sécurité, audit & conformité .....               | 12 |
| 09 | Scalabilité & performance .....                  | 13 |
| 10 | Décisions architecturales (ADR) .....            | 14 |
| 11 | Comparatif Standard vs Premium .....             | 16 |
| 12 | Roadmap & dette technique .....                  | 17 |

# 01 Contexte & objectifs architecturaux

LeadFarm remplace la tenue de registres phytosanitaires papier (formulaires FOR.PR6.003 / FOR.PR6.004) par un écosystème numérique temps réel, assisté par IA et certifiable, pour des vergers à l'échelle industrielle.

## 1.1 — Forces directrices

L'architecture est dictée par quatre contraintes non négociables propres au marché agricole algérien et au modèle SaaS :

- **Multi-tenancy économique** — un seul socle Supabase doit servir l'objectif de 50 unités agricoles sous l'ombrellle Groupe Lachhab, sans dupliquer l'infrastructure par client.
- **Connectivité variable** — le terrain agricole a une couverture 4G intermittente ; l'app Chauffeur doit fonctionner offline-first puis synchroniser.
- **Conformité certifiable** — GLOBALG.A.P., HACCP et l'Arrêté n°1275 exigent une traçabilité immuable et auditable de chaque traitement.
- **Monétisation par paliers** — deux plans (Standard / Premium) doivent partager le même code tout en activant/désactivant des capacités de façon sûre.

## 1.2 — Attributs de qualité ciblés

| Attribut            | Cible                                | Mécanisme architectural                      |
|---------------------|--------------------------------------|----------------------------------------------|
| Isolation           | Étanchéité totale inter-tenant       | Row Level Security PostgreSQL + claim JWT    |
| Sécurité IoT        | Aucune donnée capteur falsifiable    | Signature HMAC-SHA256 vérifiée à l'ingestion |
| Auditabilité        | Traçabilité immuable des traitements | SCD2 versioning + Audit Log triggers         |
| Scalabilité données | ~260M lignes IoT/an à 50 sites       | Partitioning mensuel + vues agrégées         |
| Latence spatiale    | < 50ms sur requêtes géo              | PostGIS + index GiST                         |
| Time-to-market      | Déploiement pilote rapide            | Monolithe modulaire, pas de microservices    |

### PRINCIPE DIRECTEUR

La sécurité et la séparation des plans sont garanties **par défaut au niveau de l'infrastructure** (base de données, JWT), jamais par la seule discipline applicative. Un oubli de vérification côté code ne doit jamais ouvrir une faille de données ou de plan.

## 02 Vue d'ensemble — les 6 couches

Le système est structuré en six couches imbriquées. Chaque couche ne communique qu'avec ses voisins immédiates, ce qui limite la surface de couplage et facilite le remplacement isolé d'un composant.

| # | Couche             | Responsabilité                                             | Technologies                                           |
|---|--------------------|------------------------------------------------------------|--------------------------------------------------------|
| 1 | Clients & Devices  | Consommation et capture de données par humains et machines | Next.js Web, Capacitor mobile, ESP32, page QR publique |
| 2 | Edge & API Gateway | Rendu serveur, routage, authentification d'entrée          | Vercel CDN, RSC, Server Actions, Middleware            |
| 3 | Plan Gate          | Application unique et centralisée des plans                | JWT claim, RLS, Quota Engine, UI PlanGuard             |
| 4 | Services métier    | Logique applicative et orchestration                       | Supabase Edge Functions (Deno)                         |
| 5 | Données            | Persistance, intégrité, requêtes spatiales                 | PostgreSQL 15, PostGIS 3.3                             |
| 6 | Transverse         | Sécurité, audit, observabilité, temps réel                 | SCD2, Audit Log, RLS, Realtime WebSockets              |

### 2.1 — Choix du style architectural

Le système adopte un **monolithe modulaire couplé à des fonctions Edge**, et non une architecture microservices. Ce choix est délibéré : à l'échelle d'un pilote et d'un objectif de 50 clients, le découpage en microservices introduirait une latence réseau inter-services, une complexité opérationnelle (orchestration, service mesh, observabilité distribuée) et une dette de coordination disproportionnées par rapport au gain.

Les frontières de modules sont matérialisées au niveau du code (dossiers par domaine) et de la base (schémas logiques par domaine), ce qui préserve la possibilité d'extraire ultérieurement un domaine en service autonome si un goulot d'étranglement apparaît — par exemple le pipeline IA si le volume d'inférences explose.

#### ÉVOLUTIVITÉ DU STYLE

Le monolithe modulaire est une étape, pas une fin. Les coutures sont posées dès maintenant (séparation domaine, files de messages) pour permettre une extraction de service sans réécriture majeure.

## 03 Stratégie multi-tenant & isolation

Toutes les exploitations partagent une base de données unique. L'isolation est garantie au niveau le plus bas possible — la base de données elle-même — via Row Level Security.

### 3.1 — Modèle de cloisonnement

Chaque table du domaine porte une clé étrangère `exploitation_id`. À la connexion, un Edge Function `auth-hook` injecte ce `exploitation_id` ainsi que le `role` et le `plan_id` dans les claims du JWT de l'utilisateur. Les politiques RLS lisent directement `auth.jwt()` pour filtrer chaque ligne.

```
-- Politique RLS type : isolation stricte par exploitation
CREATE POLICY tenant_isolation ON evenement_agronomique
  USING (
    exploitation_id = (auth.jwt() ->> 'exploitation_id')::int
  );

-- Conséquence : même avec un accès API direct et un token valide,
-- un utilisateur ne peut JAMAIS lire les données d'un autre tenant.
```

### 3.2 — Stratégie d'évolution de la tenancy

| Palier client               | Stratégie                  | Justification                                             |
|-----------------------------|----------------------------|-----------------------------------------------------------|
| Standard / Premium < 500 ha | Schéma partagé + RLS       | Coût infrastructure minimal, isolation logique suffisante |
| Grand compte > 500 ha       | Schéma Postgres dédié (V2) | Isolation physique, quotas de performance garantis        |
| Coopérative (10+ fermes)    | Multi-tenant imbriqué      | Une coopérative = un tenant parent, fermes = sous-entités |

#### RISQUE MAÎTRISÉ

Le schéma partagé concentre le risque : une faille RLS exposerait potentiellement tous les tenants. Mitigation : les politiques RLS sont testées par une suite d'intégration dédiée (test-writer subagent) qui tente activement des accès cross-tenant et doit échouer.

## 04 Plan Gate — le keystone Standard / Premium

Le Plan Gate est la décision architecturale la plus structurante du système. Au lieu de disperser des conditions de plan dans tout le code, un claim JWT unique propage le plan à quatre niveaux distincts.

### 4.1 — Propagation à 4 niveaux

| Niveau         | Mécanisme                                                            | Ce qu'il protège                                                    |
|----------------|----------------------------------------------------------------------|---------------------------------------------------------------------|
| JWT Claim      | <code>plan_id</code> signé à la connexion via <code>auth-hook</code> | Source de vérité unique, infalsifiable car signée                   |
| RLS plan-aware | Politiques SQL lisant <code>auth.jwt()</code>                        | Accès aux tables Premium ( <code>lot_qr</code> , données satellite) |
| Quota Engine   | Table <code>usage_quota</code> + trigger                             | Limites Standard (10 décisions/mois, 5 capteurs)                    |
| UI PlanGuard   | Composant React conditionnel                                         | Expérience : verrouille visuellement les modules Premium            |

#### POURQUOI LE JWT ET PAS UNE SIMPLE COLONNE EN BASE ?

Lire le plan depuis la base à chaque requête créerait une dépendance et une latence supplémentaires. Le claim JWT, signé et vérifié cryptographiquement, transporte le plan sans appel base additionnel. Il est régénéré à chaque changement de plan via un refresh de session.

### 4.2 — Moteur de quota (cœur du plan Standard)

La table `usage_quota` stocke, par exploitation et par mois, les compteurs de consommation. Un trigger Postgres incrémente le compteur et bloque l'opération si le quota est dépassé.

```

-- Trigger de quota sur les décisions IA (plan Standard)
CREATE FUNCTION fn_check_quota_decision() RETURNS trigger AS $$
DECLARE q_used int; q_max int; plan text;
BEGIN
    SELECT plan_id INTO plan FROM exploitation
        WHERE id = NEW.exploitation_id;

    IF plan = 'premium' THEN RETURN NEW; -- illimité
    END IF;

    SELECT decisions_utilisees, decisions_max INTO q_used, q_max
        FROM usage_quota
        WHERE exploitation_id = NEW.exploitation_id
            AND mois = to_char(now(), 'YYYY-MM');

    IF q_used >= q_max THEN
        RAISE EXCEPTION 'QUOTA_DEPASSE: passez en Premium';
    END IF;

    UPDATE usage_quota SET decisions_utilisees = decisions_utilisees + 1
        WHERE exploitation_id = NEW.exploitation_id
            AND mois = to_char(now(), 'YYYY-MM');

    RETURN NEW;
END; $$ LANGUAGE plpgsql;

```

## 4.3 — Chemin de montée en plan (upgrade path)

Le passage Standard → Premium est conçu sans friction et sans migration de données :

- ▶ Quota atteint → modal d'upgrade in-app déclenché par l'exception `QUOTA_DEPASSE`.
- ▶ Paiement validé → `plan_id` mis à jour en base.
- ▶ Session rafraîchie → nouveau JWT avec `plan_id = premium`.
- ▶ RLS relâchée automatiquement, fonctions Premium activées, quotas ignorés. Aucune donnée déplacée.

## 05 Couche données & modèle pivot

~35 entités réparties en 5 domaines, sur PostgreSQL 15 avec extension PostGIS. L'entité `evenement_agronomique` est le pivot central autour duquel gravite toute la logique métier.

### 5.1 — Organisation par domaine

| Domaine       | Entités principales                                                                       | Particularité                                 |
|---------------|-------------------------------------------------------------------------------------------|-----------------------------------------------|
| Tenant        | exploitation, utilisateur, role, permission, tenant_utilisateur, usage_quota              | RBAC à 7 rôles, jonction utilisateur ↔ tenant |
| Géospatial    | zone, parcelle, micro_zone, donnee_satellite, donnee_meteo                                | Polygones PostGIS (SRID 4326), index GiST     |
| IoT           | capteur, type_mesure, mesure_iot, mesure_agreee, maintenance_capteur                      | Table la plus volumineuse, partitionnée       |
| Agronomique   | campagne, plantation, evenement_agronomique, protocole, etape_protocole, maladie, produit | Pivot central + référentiels                  |
| IA & Business | decision, resultat, apprentissage, recolte, revenu, depense, alerte, lot_qr               | Boucle d'apprentissage + finance              |

### 5.2 — L'entité pivot : evenement\_agronomique

Chaque acte agronomique (irrigation, traitement, récolte, détection) est un événement. Cette table relie huit autres entités, ce qui en fait le point critique en intégrité et en volume.

|                       |                                     |
|-----------------------|-------------------------------------|
| evenement_agronomique |                                     |
| ├→ evenement_maladie  | (si détection)                      |
| ├→ evenement_produit  | (si traitement phyto)               |
| ├→ depense            | (coût intervention)                 |
| ├→ recolte            | (si récolte)                        |
| ├→ resultat           | (mesure d'impact 1:N dans le temps) |
| ├→ alerte             | (notification)                      |
| ├→ decision           | (via decision_evenement, N:M)       |
| └→ audit_log          | (traçabilité auto par trigger)      |

### 5.3 — Décisions de modélisation clés (post-critique v2)

- **TYPE EVENEMENT en table de référence** — remplace un champ string libre, avec booléens de validation (`necessite_recolte`, `necessite_produit`) appliqués côté contrainte.
- **Cardinalité RESULTAT 1:N** — un événement génère plusieurs mesures dans le temps (J+7, J+30, rendement final), corrigé depuis le 1:1 initial.
- **features\_utilisees en JSONB** — snapshot immuable du contexte d'inférence IA, indexé par GIN. Cohérent avec les pratiques de ML tracking (MLflow).
- **Dates relatives dans ETAPE\_PROTOCOLE** — jours depuis plantation (J+n) au lieu de dates absolues, rendant les protocoles réutilisables d'une campagne à l'autre.



- **CULTURE + VARIETE normalisées** — élimine les incohérences de saisie ("Roma" vs "roma "), exigence GLOBALG.A.P.

## 5.4 — Stratégie de volumétrie

**5.2M**

lignes/an  
(1 site, 10 capteurs)

**260M**

lignes/an  
(50 sites)

**<50ms**

requêtes  
spatiales

**12**

partitions  
mensuelles/an

### PARTITIONING OBLIGATOIRE

`mesure_iot` est partitionnée par `RANGE (horodatage)` mensuel. Une vue matérialisée `mesure_agreee` (horaire/journalière) alimente les dashboards sans scanner les lignes brutes. Les partitions de plus de 18 mois sont purgées après agrégation.

## 06 Services métier (Edge Functions)

Neuf fonctions serverless Deno co-localisées avec la base. Chacune vérifie le `plan_id` en entrée avant exécution, appliquant le gating au niveau du service.

| Fonction                     | Déclencheur    | Rôle                                                                                        | Plan        |
|------------------------------|----------------|---------------------------------------------------------------------------------------------|-------------|
| <code>ingest-mesure</code>   | POST ESP32     | Vérifie HMAC-SHA256 → contrôle quota capteurs → insère mesure                               | 2 plans     |
| <code>auth-hook</code>       | Post-connexion | Injecte <code>plan_id</code> , <code>role</code> , <code>exploitation_id</code> dans le JWT | 2 plans     |
| <code>moteur-decision</code> | Trigger DB     | Règles IPM + KNN historique → propose décisions                                             | limité Std  |
| <code>detect-maladie</code>  | Upload photo   | Image → Hugging Face → diagnostic 38 classes                                                | limité Std  |
| <code>pull-meteo</code>      | Cron horaire   | Récupère Open-Meteo par centroïde de zone                                                   | 2 plans     |
| <code>pull-satellite</code>  | Cron 6h        | NDVI/NDWI Sentinel-2 par parcelle                                                           | Premium     |
| <code>envoyer-alerte</code>  | Trigger DB     | Email (2 plans) · SMS + WhatsApp (Premium)                                                  | multi-canal |
| <code>creer-lot-qr</code>    | Server Action  | Génère page publique de traçabilité par lot                                                 | Premium     |
| <code>generer-rapport</code> | Server Action  | PDF FOR.PR6.003/004 vers Storage                                                            | 2 plans     |

### 6.1 — Garde de plan en entrée de fonction

```
// pull-satellite : sort immédiatement pour les tenants Standard
const { plan_id } = decodeJwt(req.headers.authorization);

if (plan_id !== 'premium') {
  // Aucun appel à Sentinel Hub → économie de coût API directe
  return new Response('feature_premium_required', { status: 403 });
}
```

#### BÉNÉFICE SECONDAIRE DU GATING EDGE

Le contrôle de plan dans `pull-satellite` évite des appels coûteux à l'API Sentinel Hub pour les tenants Standard. Le gating ne protège pas seulement la fonctionnalité — il optimise directement les coûts d'infrastructure.

## 07 Flux de données critiques

Trois flux structurent l'essentiel de la valeur du système. Leur compréhension est nécessaire pour raisonner sur la performance et la fiabilité.

### 7.1 — Flux IoT → Décision IA

```
ESP32 (terrain)
→ POST signé HMAC → ingest-mesure (vérif. signature + quota)
→ MESURE_IOT (insertion brute, partition du mois)
→ Agrégation horaire/journalière (MESURE_AGREGEE)
→ Pipeline features (NDVI + météo + humidité + stress)
→ moteur-decision (IPM + KNN)
→ DECISION (recommandation + score + features_utilisees)
→ EVENEMENT_AGRONOMIQUE (intervention validée)
→ RESULTAT (mesure d'impact)
→ APPRENTISSAGE (boucle de feedback)
```

#### POINT DE FIABILITÉ CRITIQUE

L'ingestion directe ESP32 → INSERT est vulnérable aux pics de charge (50 capteurs simultanés). **Recommandation P0** : insérer une file de messages ( `pgmq` , natif Postgres) entre l'Edge Function et l'INSERT, avec un worker de drainage. Évite la perte de mesures lors d'un spike de latence.

### 7.2 — Flux économique (P&L)

```
EVENEMENT_AGRONOMIQUE
→ DEPENSE (main-d'œuvre, intrants, carburant)
→ RECOLTE (quantité, qualité, lot)
→ REVENU (quantité × prix_marché)

P&L par plantation = Σ(REVENU) - Σ(DEPENSE)
P&L par campagne = Σ(P&L des plantations)
```

### 7.3 — Flux de traçabilité (double système)

```
Modification table critique
→ Trigger Postgres
→ SCD2 versioning (nouvelle version, validité) [Premium]
→ AUDIT_LOG (diff JSON + user + IP) [2 plans]
```

## 08 Sécurité, audit & conformité

La traçabilité repose sur deux systèmes complémentaires aux responsabilités strictement distinctes — résolution d'un conflit architectural identifié en revue.

### 8.1 — SCD2 vs Audit Log : séparation des responsabilités

| Critère           | SCD2 Versioning                                   | Audit Log                                             |
|-------------------|---------------------------------------------------|-------------------------------------------------------|
| Question répondue | Qui a changé quel traitement, et pourquoi ?       | Qui s'est connecté, depuis quelle IP, quelle action ? |
| Nature            | Versioning métier agronomique                     | Traçabilité sécurité système                          |
| Mécanisme         | <code>fn_scd2_versioning()</code> sur 11 tables   | Triggers + diff JSON + SHA-256                        |
| Colonnes          | numero_version, date_validite, motif_modification | anciennes/nouvelles_valeurs, adresse_ip, checksum     |
| Conformité        | GLOBALG.A.P, HACCP, Arrêté 1275                   | ISO 27001, SOC2, RGPD                                 |
| Plan              | Premium                                           | 2 plans                                               |

#### POURQUOI DEUX SYSTÈMES ET NON UN SEUL ?

SCD2 conserve l'historique complet des *versions métier* (chaque traitement modifié devient une nouvelle ligne consultable en time-travel). L'Audit Log enregistre les *opérations système* sans versionner la donnée. Les triggers sont conçus pour ne pas se déclencher mutuellement, évitant tout conflit en production.

### 8.2 — Défense en profondeur

- **Couche réseau** — HTTPS partout, rate limiting par device\_id sur les Edge Functions exposées.
- **Couche IoT** — signature HMAC-SHA256 vérifiée avant toute insertion ; un capteur défaillant ne peut pas injecter de fausses mesures.
- **Couche données** — RLS par exploitation, jamais de filtrage applicatif seul.
- **Couche audit** — Audit Log écrit uniquement par triggers, jamais directement par l'application.

## 09 Scalabilité & performance

### 9.1 — Points de tension identifiés (priorisés)

| Priorité | Risque                                             | Mitigation                                    |
|----------|----------------------------------------------------|-----------------------------------------------|
| P0       | Explosion volumétrique MESURE_IOT (260M lignes/an) | Partitioning mensuel + purge post-agrégation  |
| P0       | Perte de mesures sur pic de charge IoT             | File pgmq + worker de drainage                |
| P1       | Flood Edge Functions par capteur défaillant        | Rate limiting par device_id                   |
| P1       | Conflit de triggers SCD2 / Audit Log               | Séparation explicite des tables et conditions |
| P2       | Requêtes JSONB lentes sur features_utilisees       | Index GIN                                     |
| P2       | Sync offline conflictuelle (Chauffeur)             | Stratégie de merge avec détection de conflits |
| P3       | Scans lourds pour dashboards                       | Vues matérialisées MESURE_AGREGEE             |

### 9.2 — Stratégie offline-first (mobile Chauffeur)

L'app Chauffeur doit fonctionner en zone sans réseau. Workbox + IndexedDB (Dexie.js) assurent le cache local. La vraie complexité est la **réconciliation** : quand le réseau revient après plusieurs saisies hors-ligne, un mécanisme de synchronisation avec détection de conflits (timestamps + résolution last-write-wins par champ) est nécessaire. Dexie.js seul ne le résout pas.

## 10 Décisions architecturales (ADR)

Les décisions structurantes sont consignées au format Architecture Decision Record pour tracer le raisonnement et permettre une remise en question future éclairée.

### ADR-001

#### Plan gating par claim JWT unique

Statut : **Accepté**

**Contexte** : Deux plans doivent partager le même code tout en activant/désactivant des capacités de manière sûre.

**Décision** : Le plan est un claim JWT signé, propagé à 4 niveaux (RLS, Edge, Quota, UI). Pas de logique de plan dispersée.

**Conséquence** : Sécurité par défaut au niveau base. Régénération du JWT requise au changement de plan.

### ADR-002

#### Monolithe modulaire plutôt que microservices

Statut : **Accepté**

**Contexte** : Pilote + objectif 50 clients. Équipe réduite.

**Décision** : Monolithe Next.js modulaire + Edge Functions. Frontières de domaine matérialisées sans découpage réseau.

**Conséquence** : Latence et complexité ops minimales. Extraction de service possible plus tard via les coutures posées.

### ADR-003

#### Double système d'audit (SCD2 + Audit Log)

Statut : **Accepté**

**Contexte** : Conflit identifié entre versioning métier et traçabilité sécurité.

**Décision** : Conserver les deux avec responsabilités strictement disjointes. SCD2 = versions métier (Premium), Audit Log = sécurité (2 plans).

**Conséquence** : Conformité agricole ET sécurité couvertes. Triggers conçus pour ne pas se déclencher mutuellement.

### ADR-004

#### Isolation par RLS plutôt que par schéma

Statut : **Accepté**

**Contexte** : Multi-tenancy économique pour 50 clients.

**Décision** : Schéma partagé + RLS pour <500ha. Schéma dédié réservé aux grands comptes en V2.

**Conséquence** : Coût infra minimal. Risque concentré sur la qualité des politiques RLS, mitigé par tests cross-tenant.

## 11 Comparatif Standard vs Premium

Synthèse des capacités activées par plan, alignée sur les mécanismes de gating décrits en section 04.

| Capacité                | Standard               | Premium                | Mécanisme de gate           |
|-------------------------|------------------------|------------------------|-----------------------------|
| Parcelles               | 5 max                  | Illimitées             | RLS + quota                 |
| Utilisateurs            | 3 max                  | Illimités              | Quota                       |
| Capteurs IoT            | 5 max                  | Illimités              | Quota (ingest-mesure)       |
| Décisions IA / mois     | 10                     | Illimitées             | Trigger usage_quota         |
| Vision IA / mois        | 10                     | Illimitée              | Quota (detect-maladie)      |
| Satellite NDVI/NDWI     | Non                    | Quotidien              | Edge guard (pull-satellite) |
| Alertes                 | Email seul             | SMS + WhatsApp + Email | Edge (envoyer-alerte)       |
| Boucle apprentissage IA | Non                    | Oui                    | RLS                         |
| Audit SCD2 time-travel  | Non                    | Complet                | Triggers conditionnels      |
| Traçabilité QR lot      | Non                    | Oui                    | RLS (lot_qr) + Edge         |
| Checklist GLOBALG.A.P   | Non                    | Temps réel             | RLS                         |
| API export / ERP        | Non                    | JSON/CSV               | RLS + API route             |
| Support                 | Email 48h              | Dédié SLA <4h          | Opérationnel                |
| <b>Tarif indicatif</b>  | <b>18 000 DZD/mois</b> | <b>42 000 DZD/mois</b> | —                           |

## 12 Roadmap & dette technique

### 12.1 — Dette technique à résorber (par priorité)

- ▶ **P0** — Partitioning de `mesure_iot` et file pgmq pour l'ingestion IoT.
- ▶ **P1** — Rate limiting Edge Functions ; clarification des triggers SCD2/Audit.
- ▶ **P1** — Normalisation de `variete` en entité propre.
- ▶ **P2** — Index GIN sur `features_utilisees` ; stratégie de merge offline.
- ▶ **P3** — Vues matérialisées `mesure_agreee` .

### 12.2 — Roadmap produit V2 (post-pilote)

| Fonctionnalité                 | Déclencheur                      | Échéance |
|--------------------------------|----------------------------------|----------|
| Missions drone (DJI Agras)     | Partenariat signé                | Q1 2027  |
| Auto-guidage RTK GPS           | Hardware acquis                  | Q2 2027  |
| IA à l'échelle de l'arbre      | 2K+ images aériennes labellisées | Q3 2027  |
| Marketplace B2B                | 10+ exploitations onboardées     | Q4 2027  |
| Tenancy hybride (schéma dédié) | Premier client >500ha            | Q1 2027  |

#### VERDICT ARCHITECTURAL

Pour un projet de niveau M2 couplé à un pilote réel, l'architecture est ambitieuse et cohérente : RLS, SCD2, HMAC sur l'IoT et gating par JWT placent LeadFarm au-dessus du standard de l'agritech nord-africaine. Les risques résiduels sont concentrés sur la volumétrie IoT (P0) et se traitent en itération sans réécriture.